

contract-review-openclaw

项目技术路线（whole-project technical roadmap）

liangch97

2026-05-07 文档版本 v1.1

Abstract

本文描述 `contract-review-openclaw` 项目的整体技术路线：从一份合同 `.docx` 到行政老师 IM 中收到的 行政版 *PDF* 报告的端到端管线，包括 OpenClaw skill 体系、入口编排、合同读取与字段抽取、规则引擎、学校范本对照、企业工商核验、报告渲染与 IM 交付、数据契约与部署形态。文档读者：项目维护者、二开者、运维。

Contents

1	项目概述	3
1.1	背景与目标	3
1.2	用户与角色	3
1.3	安全与边界	3
2	系统架构总览	4
2.1	端到端管线	4
2.2	组件清单	5
3	入口层：三种触发方式	5
3.1	IM（飞书）：默认触发	5
3.1.1	full_qcc_review.py 编排器	6
3.2	命令行：run_contract_review.py	6
3.3	Dashboard / Demo：可选	7
4	OpenClaw skill 体系	7
4.1	两个 skill 的分工	7
4.2	为何要拆 skill	7
4.3	安装机制	7
4.4	Onboarding 插件（TypeScript）	8

5 核心管线	8
5.1 合同读取 (contract_loader.py)	8
5.2 字段抽取 (contract_extractor.py)	9
5.3 规则引擎 (contract_rule_checker.py)	9
5.3.1 规则字段	10
5.3.2 check_type 取值与语义	10
5.3.3 决策映射 _map_decision	10
5.3.4 企业核验融合	11
5.4 报告写出 (contract_report_writer.py)	11
5.5 行政 PDF 渲染 (admin_report.py, 外置)	11
6 学校范本对照子系统	11
6.1 背景：单门槛失败案例	11
6.2 双门槛设计	12
6.3 使用说明剥离	12
6.4 Clauses 级 diff	12
6.5 范本库	13
6.6 回归基准	13
7 企业核验子系统	13
7.1 材料来源 (白名单)	13
7.2 QCC 流程 (demos/qcc_login_demo.py)	13
7.3 核验匹配 (company_check_matcher.py)	14
7.4 失败降级路径	14
8 数据契约	14
8.1 contract_findings_<hash>.json 顶层 schema	14
8.2 template_match 块	15
8.3 规则 YAML 字段	16
9 部署形态	16
9.1 本地 (Windows + WSL)	16
9.2 云端 ArkClaw	16
9.3 产物归档	16

10 测试与回归	16
11 后续路线	17

1. 项目概述

1.1 背景与目标

中山大学每年签订大量横向科研合同（技术开发、技术服务、技术咨询、技术转让、共同申请专利、共同申请专利等），这些合同需要由行政老师做形式化审核：检查合同要素是否齐全、关键条款是否合规、知识产权归属是否清楚、甲方企业是否真实存在等。该过程长期依赖人工逐条比对，工作量大、容易疏漏。

contract-review-openclaw 是面向 OpenClaw 平台的本地包，目标是：

- IM 一键审核。**行政老师在飞书里发一份合同 + 一句话「请帮我审核这个合同」，自动返回一份可直接转发给项目负责人的行政版 PDF。
- 形式化覆盖。**覆盖 41 条形式化规则，区分自动判定项 / 人工复核项 / 仅留档项。
- 学校范本对照。**识别合同是否基于中山大学合同范本起草，并指出条款级偏离。
- 企业工商核验。**通过企查查（QCC）扫码登录获取甲方真实工商信息，与合同抬头比对；无法核验时降级为人工材料路径。
- 本地优先 / 离线可用。**所有审核逻辑跑在本地，不依赖外部 LLM；企查查访问全程由用户控制。

1.2 用户与角色

角色	典型动作
行政老师	在飞书里发合同 + 触发指令；阅读最终 PDF；转发给项目负责人 / 合作方修改。 不需要 理解 JSON、规则编号或脚本。
二开 / 测试	在 PowerShell / Bash 里跑 <code>run_contract_review.py</code> 做批量回归；维护 <code>horizontal_contract_formal_rules.yaml</code> ；新增范本时复制 <code>.docx</code> 到 <code>references/templates/</code> 。
运维	部署 OpenClaw / WSL；维护飞书凭据；监控 <code>output/im_review/<timestamp>/</code> 的实际推送。

1.3 安全与边界

- 不调用外部 LLM。所有规则匹配、字段抽取、范本对比均为本地代码（正则 + `python-docx` + `difflib`）。

- 不绕过任何访问控制。企查查走标准浏览器访问；遇到验证码、WAF、登录失败、付费墙时主动降级到「人工材料 / 退回普通审核」。
- 企业信息白名单。甲方信息只能来自三种受控来源：本次 QCC 扫码、人工 `--company-check` 文件、用户提供的截图 / 记录。不允许从模型记忆或公开知识填充。
- 结果可解释。每条 finding 必须给出 `rule_id + evidence + location`；最终面向老师的 PDF 隐去技术名词，按「问题 / 关联条款 / 建议处理」三列展示。

2. 系统架构总览

2.1 端到端管线

图 1 给出从用户触发到 PDF 推送的完整数据流。

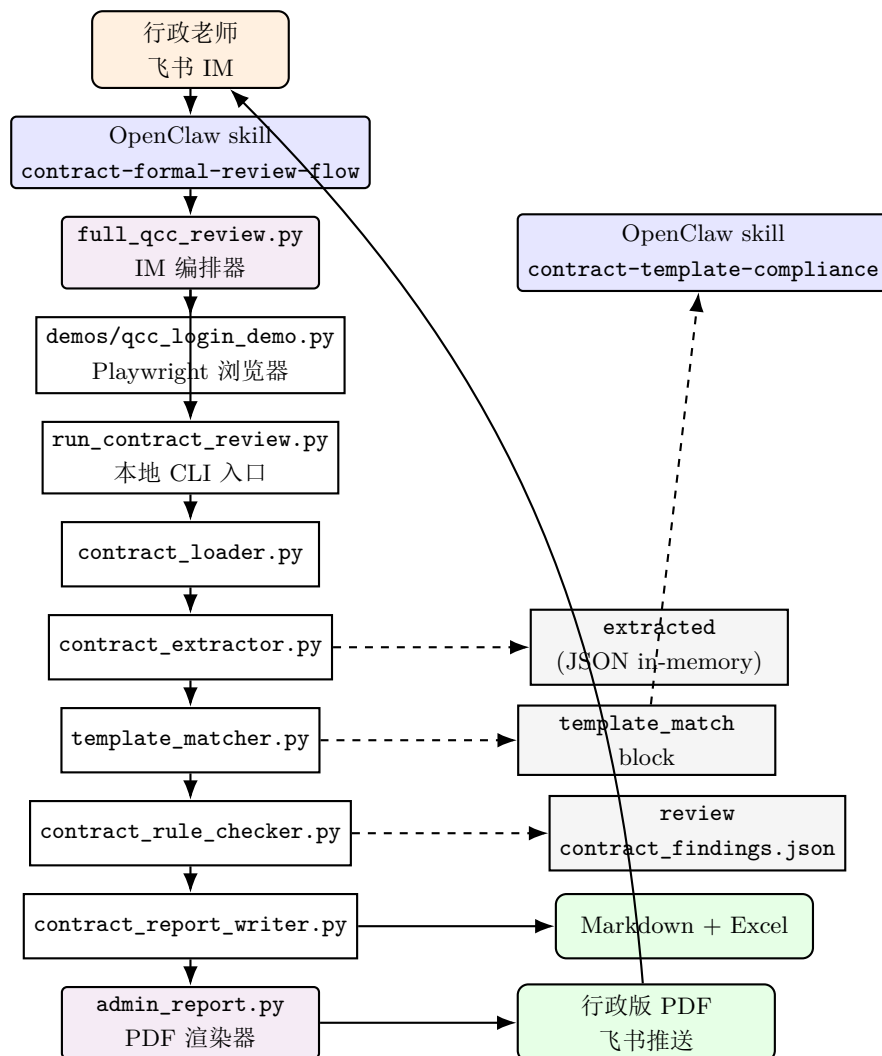


Figure 1: 端到端管线（蓝 = skill，紫 = 外置编排，白 = 管线模块，灰 = 数据，绿 = 最终交付物）。

2.2 组件清单

按角色把仓库划分为五层：

层级	目录 / 文件	职责
1. 入口	run_contract_review.py	本地 CLI 入口（Argparse）。
1. 入口	install_into_openclaw.py	把 skills/ 同步到 ~/.openclaw/skills/；写入当前解压目录路径。
1. 入口（外置）	/root/full_qcc_review.py	IM 编排器：QCC → 抽取 → 审核 → 飞书推 PDF。 代码在 OpenClaw 部署目录，不在仓库。
2. Skill	skills/contract-formal-review-openclaw	形式化审核默认入口 skill。
2. Skill	skills/contract-template-compliance/skill	范本合同对照专家/skill。
2. Skill	openclaw-extensions/Contract-review-plugin/	OpenClaw 引导插件 (TTS)
3. 管线	scripts/contract_loader.py	读取 docx/pdf/txt + 批注 + 高亮 + 表格。
3. 管线	scripts/contract_extractor.py	段落抽取（标题、项目名、金额、党事方、抬头等）。
3. 管线	scripts/contract_rule_checker.py	规则引擎。读取 41 条 YAML 规则。
3. 管线	scripts/contract_report.py	写出 Markdown / JSON / Excel 三件套。
3. 管线	scripts/template_matcher.py	学校范本对照（双门槛）。
3. 管线	scripts/company_checker.py	加载企业材料并与合同甲方匹配。
3. 管线（外置）	& company_check_matcher.py /root/admin_report.py	行政 PDF 渲染。
4. 数据	references/rules/horizontal	11 条规则 contract_formal_rules.yaml
4. 数据	references/schemas/*	三份输出 JSON Schema。
4. 数据	references/templates/0	0 份学校合同范本（部署侧）。
4. 数据	山大学 *.docx	
4. 数据	references/prompts/	给老师可复制的提示词模板。
4. 数据	references/handoff/qcc_and_arkclaw	QCC 和 飞书连接器手册。
5. 演示与测试	demos/qcc_login_demo.py	QCC 二维码登录 demo（被 full_qcc_review.py 复用）。
5. 演示与测试	demos/qcc_contract_review_demo.py	旧版扫码审核合一脚本（不推送飞书）。
5. 演示与测试	tests/	pytest 测试（规则、抽取、QCC 行为）。
5. 演示与测试	samples/	测试合同。
6. 部署	deployment/arkclaw-global	云端 ArkClaw 全局提示词。

3. 入口层：三种触发方式

3.1 IM（飞书）：默认触发

行政老师对 OpenClaw 说：

请帮我审核这个合同，<合同文件完整路径>

OpenClaw 触发 `contract-formal-review-flow` skill；该 skill 强制要求模型只走一条命令：

```
python3 /root/full_qcc_review.py "<contract_file_path>"
```

3.1.1 full_qcc_review.py 编排器

外置在 OpenClaw 部署目录的包装器（691 行）。它的责任：

1. 初步推送状态。`fs_text(" 开始审核\n 合同： ... 步骤 1/4：启动 QCC 登录...")`
2. 快速路径检测。如果检测到飞书凭据缺失或合同名像 已审核报告（避免对生成的 PDF 再做审核），直接走「无 QCC」路径，跳到第 4 步。
3. 抽取党事方甲方。`extract_party_a()` 用正则从合同抽出甲方名字；如果用户传了 `--company-name`，以参数为准。
4. QCC 扫码登录。复用 `demos/qcc_login_demo.py` 的 Playwright 浏览器；优先复用 `.browser-profiles/qcc-` 的本机会话；失效时通过 `fs_image` 把二维码截图推到飞书让老师扫码；最长等 15 分钟。
5. QCC 公司搜索。登录成功后搜索甲方公司，截屏，导出 `output/qcc_login_demo/company_check_draft_*.j`
6. 触发本地审核。

```
python run_contract_review.py "<contract>" --company-check "<draft.json>"
```

 生成 Markdown / JSON / Excel 三件套到 `output/`。
7. 渲染行政 PDF。导入 `admin_report.py`，把 `contract_findings_<hash>.json` + Markdown 渲染成 合同审核报告 _ 行政版.pdf（含「三、学校范本对照」一节）。
8. 推送 PDF。`fs_file()` 通过 `openclaw-gateway` 把 PDF 上传到飞书并发给老师。
9. 回归归档。所有产物落到 `output/im_review/<timestamp>/` 便于回查。

失败路径：QCC 验证码 / 405 / 403 / 登录超时 / 浏览器崩溃 / 飞书凭据缺失，全部包装为「降级到无 QCC 审核」并在最终 PDF 里写明「外部企业核验未完成，请补充企查查截图或人工核验记录」。

3.2 命令行：run_contract_review.py

```
# 仅本地审核（不查 QCC）
python run_contract_review.py "<contract.docx>"

# 带人工整理的企业核验材料
python run_contract_review.py "<contract.docx>" \
    --company-check "<company_check.json|csv|xlsx>" \
    --rules <rules.yaml>          # 默认 references/rules/horizontal_*.yaml
    --output-dir <dir>           # 默认 ./output
```

是 `full_qcc_review.py` 内部也调用的核心入口。返回到 `stdout`：

```
[done] decision=基本可审, 建议人工复核 confidence=0.72
[md]   ./output/contract_review.md
[json] ./output/contract_findings_<hash>.json
[xlsx] ./output/合同形式化审核总表.xlsx
```

3.3 Dashboard / Demo: 可选

- `demos/qcc_login_demo.py "<company>" --login-only --hold-seconds 900`: 仅测试 QCC 二维码登录, 保留窗口 15 分钟。
- `demos/qcc_contract_review_demo.py "<contract>" --reuse-profile`: 旧版扫码 + 审核合一脚本, 不推送飞书, 保留作为离线调试。

4. OpenClaw skill 体系

4.1 两个 skill 的分工

OpenClaw 在解析用户消息时按 skill description 中的触发词激活相应 skill。本项目当前提供两个并列 skill:

Skill	规模	触发词与职责
<code>contract-formal-review-full</code>	11.4 KB / 214 行	触发: 合同审核 / 横向合同 / 帮我审核 / 审核这个合同 / <i>SYSU contract review</i> 。强制模型只走 <code>full_qcc_review.py</code> , 保证 IM 模式下不输出文本摘要 (PDF 即交付)。
<code>contract-template-compliance</code>	7.2 KB / 99 行	触发: 范本 / 范式 / 示范文本 / 学校范本 / 范本对照 / 模板对照 / 条款偏离。读取 <code>template_match</code> JSON 字段并产出范本符合性结论; 不重复执行任何脚本。

4.2 为何要拆 skill

Anthropic Skills 设计准则之一: 每个 *SKILL.md* 单一职责, 体量 $\leq 5\text{ KB}$ 。当一个 skill 承担多个异构 workflow 时, 模型 attention 被稀释, 会出现「忘记调用工具」「混淆 IM 模式回复结构」等表现。本项目早期把范本对照塞进主 skill 后, 主 *SKILL.md* 涨到 22 KB / 277 行 (覆盖 7 类 workflow), 随即出现 skill 把握能力下降; v1.1 拆分后恢复正常。

4.3 安装机制

```
# install_into_openclaw.py 的核心逻辑
SOURCE_SKILLS = PROJECT_ROOT / "skills"
TARGET = Path.home() / ".openclaw" / "skills"
for skill_dir in SOURCE_SKILLS.iterdir():
```

```
target_dir = TARGET / skill_dir.name
shutil.copytree(skill_dir, target_dir, dirs_exist_ok=True)
# 把 SKILL.md 里的 <PROJECT_ROOT> 占位符替换为本机绝对路径
skill_md = target_dir / "SKILL.md"
skill_md.write_text(
    skill_md.read_text().replace("<PROJECT_ROOT>", str(PROJECT_ROOT))
)
```

换目录或换机器后必须重新执行，因为 SKILL.md 内含「本机解压目录」字面量，OpenClaw 模型据此选择脚本路径。

4.4 Onboarding 插件 (TypeScript)

openclaw-extensions/contract-review-onboarding/ 是一个 OpenClaw dashboard 插件，注册时机：

- openclaw.plugin.json 声明插件入口与名称。
- index.ts 在 dashboard 启动时往侧边栏插一段引导：「从 install_into_openclaw.py 安装 skill → 给老师发提示词模板 → 跑一个样例合同」三步走。

5. 核心管线

5.1 合同读取 (contract_loader.py)

支持格式：.docx（首选）/.pdf /.txt。

.docx 读取深度：

- 正文段落（paragraphs）。
- 表格（tables）：按行用 | 拼接成「行内文本」；合并单元格折叠去重。
- Word 批注：解析 word/comments.xml 拿到 reviewer 评论及其锚点条款（用于规则的 reviewer_comments 对齐）。
- 高亮文字：w:highlight 样式标注的文字单独留一份 highlights 列表，供规则引擎识别老师手动标注的风险点。

输出结构：

```
{
  "source_file": "<absolute path>",
  "ordered_blocks": [{"kind": "para"|"table", "text": "..."}],
  "comments": [{"id": "1", "anchor": "...", "text": "..."}],
  "highlights": [...],
  "full_text": "<concatenated body text>",
  "warnings": [...]}
}
```


5.2 字段抽取 (contract_extractor.py)

15.5 KB / 单文件正则 + 启发式。抽取的核心字段：

字段	抽取策略	用途
title	取前 20 行中含「合同 / 协议 / 技术 / 合作」的最长行	报告标题、QCC 关键词候选
contract_no	「合同编号 / 编号 / 编号:」正则	报告抬头
sign_date	/ 「签订日期 / 签署日期 / 签订地点」正则	形式化规则
sign_place		
party_a_name	/ _PARTY_ROLE_ALIASES (甲方/委托方/转让方/受让方/受托方/合作方/服务方/需求方/供方) 映射到 甲 / 乙, 再正则取冒号后的非空文本	QCC 搜索关键词、规则匹配
party_b_name		
project_name	标题 + 行内「项目名称」字段	报告
contract_type	标题与正文关键词分类 (开发 / 服务 / 咨询 / 转让 / 共同申请专利)	规则路由 (不同合同走不同规则集)
amount	「金额」/「¥」正则 + 中文大写	形式化规则
ip_provisions	「知识产权 / 专利 / 著作权 / 背景知识产权」关键词聚合	知识产权摘要
template_match	调用 <code>template_matcher.detect_template_match</code> (包装在 <code>_detect_template_match_safe</code> 里)	范本对照下游消费

标题正则升级 (v1.1): 原 `HEADING_RE` 只识别 第 N 条 / 阿拉伯数字 / 附件; 扩展后增加 一、 / (一) / 第一部分 / 阿拉伯数字 + . 多级编号, 覆盖学校范本的「使用说明」段。

5.3 规则引擎 (contract_rule_checker.py)

36.2 KB。规则文件 `references/rules/horizontal_contract_formal_rules.yaml` 共 41 条规则, 分布在若干 `section` (其他条款 / 系统审核要点 / 当事人信息 / 合同要素 / 知识产权 / 保密 / 违约 / 争议解决 / 经费等)。

5.3.1 规则字段

```
- id: F4
  section: 其他条款
  title: 论文条款合规
  scope: enforce                # enforce | review | not_checked
  severity: warn                # block | warn | info
  source_text: "不得约定代写论文、论文单位或作者排名"
  comment_guidance: "搜索论文排名关键词"
  reviewer_comments:
    - {id: 28, anchor: "...", text: "..."}
  check_type: forbidden_phrase_any
  patterns: ["论文排名", "作者排名", "第一作者", "通讯作者",
            "论文单位", "代写论文", "帮忙写论文", "撰写论文"]
  message: "检测到疑似不合规论文条款"
  suggestion: "删除论文代写或作者排名约定"
```

5.3.2 check_type 取值与语义

check_type	语义
party_name	/ 党事方姓名 / 抬头是否完整、非空
basic_parties	
contacts_required	联系信息（电话 / 邮箱 / 地址）是否齐全
contact_email_domain	联系邮箱域名是否合规
phrase_any	命中任一关键词 → 触发 finding
forbidden_phrase_any	命中任一关键词 → 触发 违规 finding（如论文代写）
required_phrase	必须出现某条款，否则触发缺失 finding
bank_account	收款账户合规检查（账号、户名、行号格式）
payment_first_installment	首付款比例是否合规
performance_budget	绩效与经费比例
ip_ownership	知识产权归属是否清楚
shared_ip_distribution	共有知识产权份额是否 $\geq 1/N$
confidentiality_term	保密期限合规
review_only	仅人工复核（不参与自动判定）
not_checked	仅留档（不计入风险数）

5.3.3 决策映射 _map_decision

$$\text{decision} = \begin{cases} \text{「存在重大问题，需修改/补充」} & \exists f.\text{severity} = \text{block} \\ \text{「基本可审，建议人工复核」} & \exists f \\ \text{「未发现形式化问题」} & \text{otherwise} \end{cases} \quad (1)$$

对应 confidence: 0.9 / 0.72 / 0.95。

5.3.4 企业核验融合

当传入 `company_check` 参数时，规则引擎额外执行：

- 党事方甲方名 vs 工商登记名（一致 / 字面差异 / 完全不一致）。
- 关联关系（与中山大学是否存在关联）。
- 民用 / 非涉密 / 非军工属性。

未提供材料时，这些项作为「人工复核事项」留档。

5.4 报告写出 (`contract_report_writer.py`)

12.7 KB，三件套同步写出：

- `output/contract_review.md` ——行政摘要 Markdown。包含「审核结论」「建议先修改的事项」「需要人工确认或补充材料的事项」「企业信息核验情况」「知识产权重点」五节，全部用 Markdown 表格。
- `output/contract_findings_<hash>.json` ——结构化结果,遵循 `references/schemas/contract_review`. (见第 8 节)。
- `output/合同形式化审核总表.xlsx` ——多 sheet 的 Excel: `findings` / `not_checked` / 党事方信息 / 抽取字段。

5.5 行政 PDF 渲染 (`admin_report.py`, 外置)

部署在 OpenClaw 主机的 `/root/admin_report.py` 用 `weasyprint` 把 `contract_findings_<hash>.json` 渲染为 A4 PDF 合同审核报告 _ 行政版.pdf，结构：

1. 审核结论 + 决策与置信度
2. 建议先修改的事项（表格）
3. 学校范本对照（基于 `template_match` 字段）
4. 需要人工确认 / 补充材料的事项
5. 企业信息核验情况
6. 知识产权重点

6. 学校范本对照子系统

6.1 背景：单门槛失败案例

v1.0 的 `template_matcher.py` 只用 `difflib.SequenceMatcher` 对「合同正文 vs 范本正文（去波动 token 后）」算相似度，超过 $\theta = 0.40$ 即判定为「基于学校范本」。失败案例：行业起草

的技术服务合同（乙方为中山大学），实测相似度 ≈ 0.56 ，被误判为基于 中山大学技术服务合同.docx；进一步使「使用说明」段被切成「条款」，导致行政 PDF 第三节冒出 28 个 removed + 32 个 added 噪声。

6.2 双门槛设计

新版判定：形状信号与内容信号同时通过：

$$\text{matched} = (s_{\text{best}} \geq \theta_{\text{shape}}) \wedge (\#\text{anchor}_{\text{contract}} \geq k_{\text{anchor}}) \quad (2)$$

- $s_{\text{best}} = \max_T \text{Ratio}(\sigma(C), \sigma(T))$ ， C 是合同文本、 T 是范本文本、 σ 是 `_strip_volatile`（清掉日期 / 金额 / 占位符等易变 token）。
- $\theta_{\text{shape}} = 0.65$ 。
- `anchor` 集为学校范本独有的短语集合（见表 6）， $k_{\text{anchor}} = 1$ 。

Table 6: SYSU 范本独有锚点短语清单

保留（独有）	已剔除（中性 / 误导）
【科学研究院】	中山大学
【科研管理部门】	中大
学校合同管理信息系统	示范文本
合同管理信息系统	
中山大学 • 深圳 / 中山大学 • 深圳 / 中山大学 深圳	
本合同由【科学研究院】	
本合同由【科研管理部门】	

为什么剔除 中山大学 / 示范文本 / 中大：中大经常作为乙方出现在行业自拟合同中（这只能说明合同的乙方是中大的，并不意味着合同采用了学校范本）；示范文本还会出现在国家科技部范本里。这三个短语没有判别力。

6.3 使用说明剥离

SYSU 范本 .docx 起始有「使用说明」段（一、二、三、…七、）。`_strip_preamble` 在文本中存在第 N 条时，丢弃该处之前的所有行；否则保留原文。

6.4 Clauses 级 diff

匹配通过后做条款级对比：

1. `_split_clauses`：按 `HEADING_RE` 切条款。

2. `_match_clause_pairs`: 在范本条款与合同条款之间做匈牙利式对齐（按 `difflib.ratio` 最大化）。
3. 标注 5 状态: `unchanged / modified / rewritten / added / removed`。
4. 输出 `counts` 统计 + 每条条款的 `template_excerpt` 与 `contract_excerpt`（最多 120 字）。

6.5 范本库

`references/templates/` 收纳学校发布的合同范本:

```
中山大学专利技术转让合同.docx
中山大学共同申请专利合同.docx
中山大学技术咨询合同.docx
中山大学技术服务合同.docx
中山大学技术服务委托合同.docx
技术开发（委托）合同-中大为乙方.docx
```

新增范本只需把 `.docx` 放进该目录; `template_matcher.load_templates()` 在第一次调用时缓存内容。

环境变量 `CONTRACT_TEMPLATES_DIR` 可覆盖默认目录路径。

6.6 回归基准

案例	matched	相似度	anchor 命中	结论
反例：行业自拟（乙方 = 中大）	False	0.41	0	<code>non_template_contract: similarity_0.41<0.65 and anchors_0<1</code>
正例：基于专利转让范本	True	0.95	5	匹配《中山大学专利技术转让合同》；未改动 38 / 修改 9 / 改写 5 / 新增 2 / 删除 7

7. 企业核验子系统

7.1 材料来源（白名单）

1. QCC 扫码登录后由 Playwright 抓到的工商页面文字（最常用）。
2. 用户提供的 `--company-check` JSON / CSV / XLSX。
3. 用户提供的截图 / 复制文本，由测试者整理成 `--company-check` 文件。

7.2 QCC 流程 (`demons/qcc_login_demo.py`)

1. 启动 Playwright Chromium，复用本机 `.browser-profiles/qcc-demo`。

2. 打开 <https://www.qcc.com>，点击「登录」。
3. 扫二维码 → 通过 `full_qcc_review.fs_image` 推到飞书。
4. 轮询登录状态；超时（默认 15 分钟）后退出。
5. 登录成功后搜索甲方公司，截取详情页。
6. 解析详情页文字（基本信息、统一信用代码、法人、地址、状态、关联关系、风险信息），写入 `output/qcc_login_demo/company_check_draft_*.json`。

7.3 核验匹配 (`company_check_matcher.py`)

读取 `company_check` JSON 并与合同抽取出的甲方做匹配：

- `party_a_match`: 完全匹配 / 字面差异（如全角半角）/ 完全不一致。
- `related_party_check`: 是否与中山大学存在关联（公开股权 / 校办企业 / 校友企业声明）。
- `military_defense_check`: 检索是否有军工 / 涉密属性。

输出落到 `review.external_checks`，参与 `contract_rule_checker.py` 的最终 finding 与决策。

7.4 失败降级路径

- QCC 登录超时 / 验证码 / 405 / 403 → 降级为「无 QCC」审核，`external_checks.source = "manual"`，并在最终 PDF 写明「外部企业核验未完成」。
- 飞书凭据缺失（`[config] Feishu APP_ID/...`）→ 跳过 IM 推送，但其它产物正常落到 `output/`。
- 合同名疑似「已审核报告」（`looks_like_previous_report`）→ 不再触发 QCC 与重审，避免循环。

8. 数据契约

8.1 `contract_findings_<hash>.json` 顶层 schema

```
{
  "source_file": "<absolute path>",
  "decision": "未发现形式化问题|基本可审，建议人工复核|存在重大问题，需修改/补充",
  "confidence": 0.0-1.0,
  "summary_stats": {
    "findings_total": int, "risk_total": int,
    "enforce_total": int, "review_total": int,
    "info_total": int,    "not_checked_total": int,
    "rules_total": int,   "rules_hit_total": int,
    "rules_pass_total": int, "rules_not_checked_total": int,
    "by_scope": {...}, "by_severity": {...},
    "risk_rule_ids": [...], "review_rule_ids": [...], "info_rule_ids": [...]
```

```

},
"findings": [
  {
    "rule_id": "F4", "section": "其他条款",
    "severity": "block|warn|info", "scope": "enforce|review",
    "title": "...", "evidence": "...", "location": "...",
    "message": "...", "suggestion": "...",
    "confidence": 0.0-1.0,
    "details": {...}
  }
],
"not_checked_items": [...],
"rule_results": [...],
"extracted": {
  "title": "...", "contract_no": "...", "party_a_name": "...",
  "party_b_name": "...", "amount": "...", "ip_provisions": [...],
  "template_match": { ... }
},
"external_checks": {
  "company_check_provided": bool,
  "source": "qcc|manual|none",
  "checked_at": "...",
  "evidence_files": [...],
  "party_a_match": { ... },
  "related_party_check": { ... },
  "military_defense_check": { ... }
},
"generated_at": "ISO-8601"
}

```

8.2 template_match 块

```

{
  "matched": true|false,
  "template_name": "中山大学专利技术转让合同.docx",
  "similarity": 0.947,
  "anchor_hits": 5,
  "anchor_min": 1,
  "match_threshold": 0.65,
  "templates_considered": [...],
  "reason": "non_template_contract: ..." | null,
  "clauses": [
    {
      "id": "第三条", "title": "...",
      "status": "unchanged|modified|rewritten|added|removed",
      "diff_ratio": 0.72,
      "template_excerpt": "...", "contract_excerpt": "..."
    }
  ],
  "counts": {"unchanged": N, "modified": N, "rewritten": N,
    "added": N, "removed": N},
  "modified_count": N,

```

```
"summary": "..."  
}
```

8.3 规则 YAML 字段

每条规则强制字段: id, section, title, scope, severity, check_type, message。可选字段: patterns, source_text, comment_guidance, reviewer_comments, suggestion。scope 和 check_type 不一致时, scope=not_checked 优先 (仅留档不参与决策)。

9. 部署形态

9.1 本地 (Windows + WSL)

```
# Windows 侧 (命令行调试 / 批量回归)  
git clone https://github.com/liangch97/verifyAgent.git  
cd verifyAgent  
pip install -r requirements.txt  
python run_contract_review.py "<contract.docx>"  
  
# WSL 侧 (OpenClaw skill 部署)  
python install_into_openclaw.py          # 拷入 ~/.openclaw/skills/  
# 启动 openclaw-gateway / openclaw 后即可在飞书触发
```

9.2 云端 ArkClaw

deployment/arkclaw-global-prompt.md 是面向云端 ArkClaw 的全局提示词。云端不内置飞书发送逻辑——由 ArkClaw 平台读取产物 (output/contract_review.md / 合同审核报告 _ 行政版.pdf / qcc_result_*.png) 并调用平台连接器发送。

9.3 产物归档

每次 full_qcc_review.py 跑完会把所有产物归档到 output/im_review/<timestamp>/:

- contract_review.md
- contract_findings_<hash>.json
- 合同形式化审核总表.xlsx
- 合同审核报告 _ 行政版.pdf (飞书推的就是这一份)
- qcc_result_*.png (如有 QCC 成功)

10. 测试与回归

测试	覆盖
tests/test_contract_rule_checker.py	41 条规则的命中 / 不命中单测，含 check_type 各分支。
tests/test_company_check.py	工商核验加载与匹配（JSON / CSV / XLSX 三种输入）。
tests/test_qcc_login_wait_defaults.py	qcc_login_demo.py 默认 hold-seconds 行为（曾经是 0 秒导致窗口立刻关）。
tests/test_qcc_visible_text_extractor.py	QCC 详情页可见文字抽取，覆盖关键 div。
tests/test_openclaw_global_prompt_config.py	Openclaw 全局提示词配置加载与校验。
tests/smoke_extractor_cjk.py	CJK 字符下党事方姓名抽取冒烟测试。

额外的诊断脚本(v1.1 提供,根目录的 _diag_*.py / _smoketest_skills.py / _e2e_template.py, 一次性使用): 分别覆盖匹配器阈值扫描、锚点命中、条款 diff、skill YAML 元数据、端到端流水线。

11. 后续路线

- 语义化范本匹配。在 template_matcher 中加入条款级 embedding（如 BGE-small-zh）做语义相似度，与现有形状信号融合，进一步降低边缘案例（相似度 0.55~0.65）的误判风险。
- 条款级修改建议。把 clauses[].status=rewritten 的条款交给规则引擎做二次审查，生成针对性的修改建议（而不仅是「重大改写需复核法务」一句话）。
- 多范本复合识别。支持「主范本 + 附加条款范本」的复合识别（如「专利技术转让 + 共同申请专利」）。
- 规则 YAML 编辑器。给二开提供轻量 Web 编辑器，校验 scope 与 check_type 的合法组合，避免误改。
- Skill 元数据预检。install_into_openclaw.py 增加 YAML 解析 + description 长度 / 触发词覆盖度的预检，避免装入破损的 SKILL.md。
- 合成回归集。构造 30+ 条「微改写 / 重命名甲乙双方 / 调整条款顺序」的合成样本，确保 counts 在改动后仍稳定。
- 运行时观测。把 full_qcc_review.py 的步骤时间、QCC 成功率、Feishu 推送成功率上报到一个本地仪表盘。

附录 A：版本历史

- v1.0 (2026-04)。单 skill contract-formal-review-flow; 范本匹配器单门槛 $\theta = 0.40$; QCC + Feishu 链路上线; 41 条规则定型; 样例 华为-中大智算集群可靠性测评技术合作协议-脱敏版.docx。

- **v1.1** (2026-05)。拆出 `contract-template-compliance` skill；主 skill 精简至 11.4 KB；范本匹配器升级为形状 + 锚点双门槛 ($\theta = 0.65$, $\text{anchor} \geq 1$) + 使用说明剥离；QCC / ArkClaw 长篇手册外移到 `references/handoff/`；新增 `CHANGELOG.md` 与本技术路线文档。

附录 B：常用命令速查

```
# 1. 安装两个 skill
python install_into_openclaw.py

# 2. 本地审核（不查 QCC）
python run_contract_review.py "<contract.docx>"

# 3. 本地审核 + 人工核验材料
python run_contract_review.py "<contract.docx>" \
    --company-check "<company_check.json>"

# 4. IM 模式（部署侧）—— OpenClaw 自动调用
python3 /root/full_qcc_review.py "<contract>"

# 5. QCC 二维码登录 demo（仅测试登录，不查公司）
python demos/qcc_login_demo.py "<company_name>" --login-only \
    --hold-seconds 900

# 6. 重新渲染本技术路线 PDF
cd docs
xelatex -interaction=nonstopmode technical_roadmap.tex
xelatex -interaction=nonstopmode technical_roadmap.tex
```